

184.686 VU Datenbanksysteme

Transaktionen

Katja Hose

Institut für Logic and Computation

Sommersemester 2026



Informatics

Übersicht I

- 1 Transaktionen
 - Eigenschaften
 - Operationen auf Transaktionsebene
 - Umsetzung der ACID-Eigenschaften
- 2 Schedules und Serialisierbarkeit
 - Schedules
 - Conflict Serializability
 - Conflict Graphs (Precedence Graphs)
 - Recoverable Schedules und Cascadeless Schedules
- 3 Mehrbenutzersynchronisation
 - Sperrbasierte Synchronisation
 - Zwei-Phasen-Sperrprotokoll (2PL)
 - Konvertierung von Sperren
 - Erkennung von Deadlocks

Einführung

Beispiel einer typischen Bankanwendung:

- ① Lese den Kontostand von A in die Variable a : $read(A, a);$
- ② Reduziere den Kontostand um 50 Euro: $a := a - 50;$
- ③ Schreibe den neuen Kontostand in die Datenbank: $write(A, a);$
- ④ Lese den Kontostand von B in die Variable b : $read(B, b);$
- ⑤ Erhöhe den Kontostand um 50 Euro: $b := b + 50;$
- ⑥ Schreibe den neuen Kontostand in die Datenbank: $write(B, b);$

- Alle Schritte müssen als Einheit betrachtet werden
“Alles oder nichts”-Prinzip
- Sobald abgeschlossen, müssen die Änderungen permanent gespeichert werden
- ...

Was ist eine Transaktion?

Eine **Transaktion** ist eine Bündelung mehrerer Datenbankoperationen, die zusammen eine **logische Einheit** bilden. In einer Transaktion wird auf verschiedene Datenbankeinträge zugegriffen, wobei einige Einträge möglicherweise verändert werden.

Transaktionsgrenzen werden durch den Benutzer
(bzw. die Softwareanwendung) definiert!

Übersicht

- 1 Transaktionen
 - Eigenschaften
 - Operationen auf Transaktionsebene
 - Umsetzung der ACID-Eigenschaften

Operationen auf Transaktionsebene

“autocommit” Modus

Jeder Befehl wird in einer eigenen Transaktion ausgeführt.

Einfache Konsistenzprüfungen (Checks)

```
CREATE TABLE emp(  
  eid      INT          PRIMARY KEY,  
  ename    VARCHAR(30) NOT NULL,  
  salary   INT          NOT NULL CHECK (salary > 0)  
);
```

```
-- primary key violation  
insert into emp values (11, 'Kim', 200);  
-- Not null constraint violation  
insert into emp values (44, NULL, 200);  
-- Check statement violation  
insert into emp values (44, 'Kim', -200);
```

- Viele Fehler können vom DBMS erkannt werden!
- Verwenden Sie diese Checks!

Savepoints

Transaktionen von langer Dauer können zusätzlich Savepoints definieren

```
SAVEPOINT savepoint_name;
```

Definiert einen Punkt/Zustand innerhalb einer Transaktion.

Eine Transaktion kann bis zum Savepoint **teilweise rückgängig gemacht werden**.

```
ROLLBACK TO savepoint_name;
```

Setzt die aktive Transaktion zurück bis zum Savepoints savepoint_name

Beispiel

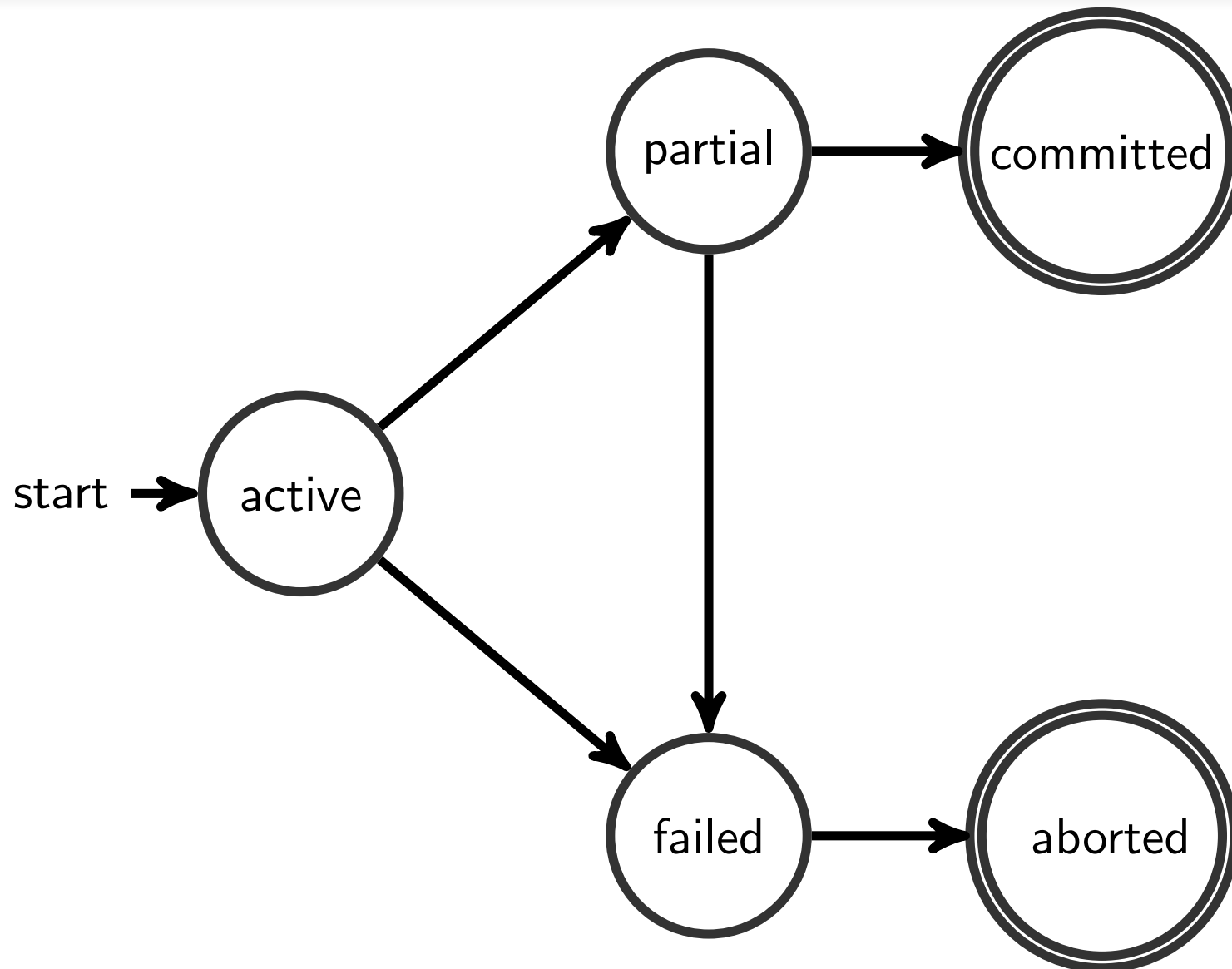
```
BEGIN;  
INSERT INTO tab VALUES...  
SAVEPOINT A;  
INSERT INTO tab VALUES...  
SAVEPOINT B;  
SELECT * FROM tab;  
ROLLBACK TO A;  
SELECT * FROM tab;  
...
```

Abschluss einer Transaktion

Für den Abschluss einer Transaktion gibt es drei Möglichkeiten:

- 1 Den **erfolgreichen** Abschluss durch **Commit**
- 2 Den **erfolglosen** Abschluss durch ein **Abort**
- 3 Den **erfolglosen** Abschluss durch einen **Fehler**

Zustandsdiagramm für Transaktionen



Übersicht

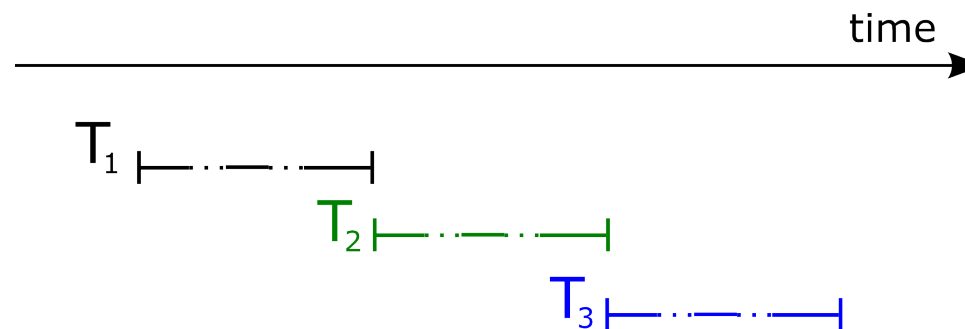
- 2 Schedules und Serialisierbarkeit
 - Schedules
 - Conflict Serializability
 - Conflict Graphs (Precedence Graphs)
 - Recoverable Schedules und Cascadeless Schedules

Nebenläufigkeit (Parallelität)

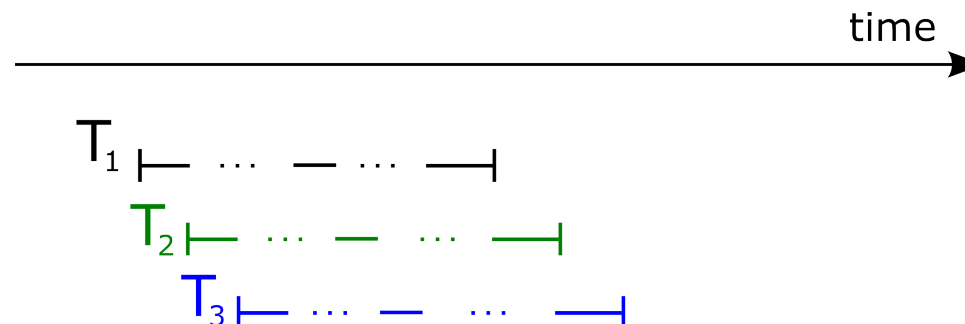
Das „I“ in ACID.

Ausführung der drei Transaktionen T_1 , T_2 , und T_3

(a) im Einzelbetrieb



(b) im (nebenläufigen) Mehrbenutzerbetrieb mit verzahnter Ausführung



Probleme bei nebenläufiger Ausführung

Lost Updates (verlorengegangene Änderungen durch Überschreiben)

Schritt	T_1	T_2
1.	read(A, a_1)	
2.	$a_1 := a_1 - 300$	
3.		read(A, a_2)
4.		$a_2 := a_2 * 1.03$
5.		write(A, a_2)
6.	write(A, a_1)	
7.	read(B, b_1)	
8.	$b_1 := b_1 + 300$	
9.	write(B, b_1)	

Probleme bei nebenläufiger Ausführung

Dirty Read (Abhängigkeit von nicht freigegebenen Änderungen)

Steps	T_1	T_2
1.	read(A, a_1)	
2.	$a_1 := a_1 - 300$	
3.	write(A, a_1)	
4.		read(A, a_2)
5.		$a_2 := a_2 * 1.03$
6.		write(A, a_2)
7.	read(B, b_1)	
8.	...	
9.	abort	

Probleme bei nebenläufiger Ausführung

Non-Repeatable Read (Abhängigkeit von anderen Updates)

T_1	T_2
update account set balance=42000 where accountID=12345	select sum(balance) from account select sum(balance) from account

Übersicht

- 2 Schedules und Serialisierbarkeit
 - Schedules
 - Conflict Serializability
 - Conflict Graphs (Precedence Graphs)
 - Recoverable Schedules und Cascadeless Schedules

Nebenläufigkeit und Korrektheit

Zentrales System mit gleichzeitigem Zugriff durch mehrere Benutzer

- Datenbank besteht aus zwei Einträgen: X und Y
- Einziges Kriterium für Korrektheit: $X = Y$
- Folgende Transaktionen:

$$\begin{array}{ll} T_1 & X \leftarrow X + 1 \\ & Y \leftarrow Y + 1 \end{array} \quad \begin{array}{ll} T_2 & X \leftarrow 2 * X \\ & Y \leftarrow 2 * Y \end{array}$$

- Initial: $X=10$ und $Y=10$.
- T_1 gefolgt von $T_2 \Rightarrow X = 22$ and $Y = 22$
- T_2 gefolgt von $T_1 \Rightarrow X = 21$ and $Y = 21$

Ein Beispiel

Wert von X: 21

Wert von Y: 21

schedule S_0	
T_1	T_2
	read(X, x) $x \leftarrow 2x$ write(X, x) read(Y, y) $y \leftarrow 2y$ write(Y, y)
read(X, x) $x \leftarrow x+1$ write(X, x) read(Y, y) $y \leftarrow y+1$ write(Y, y)	

Menti

Fragen 8–10

Beispiele:

schedule S_0		schedule $S_{0'}$		schedule S_1	
T_1	T_2	T_1	T_2	T_1	T_2
read(X, x) $x \leftarrow x+1$ write(X, x) read(Y, y) $y \leftarrow y+1$ write(Y, y)	read(X, x) $x \leftarrow 2x$ write(X, x) read(Y, y) $y \leftarrow 2y$ write(Y, y)	read(X, x) $x \leftarrow x+1$ write(X, x) read(Y, y) $y \leftarrow y+1$ write(Y, y)	read(X, x) $x \leftarrow 2x$ write(X, x) read(Y, y) $y \leftarrow 2y$ write(Y, y)	read(X, x) $x \leftarrow x+1$ write(X, x) read(Y, y) $y \leftarrow y+1$ write(Y, y)	read(X, x) $x \leftarrow 2x$ write(X, x) read(Y, y) $y \leftarrow 2y$ write(Y, y)

- $X = 21, Y = 21$

- **Serieller Schedule**

- $X = 21, Y = 21$

- **Nebenläufiger Schedule**

- $X = 22, Y = 21$

- **Ungültiger Schedule**

Menti

Frage 11

Beispiel

schedule S_2	
T_3	T_4
$\text{read}(X, x)$ $x \leftarrow x+1$ $\text{write}(X, x)$ $\text{read}(Y, y)$ $y \leftarrow y+1$ $\text{write}(Y, y)$	$\text{read}(X, x)$ $x \leftarrow 2x$ $\text{write}(X, x)$ $\text{read}(Y, y)$ $y \leftarrow 2y$ $\text{write}(Y, y)$

Initial: $X = 10$ und $Y = 10$

$\Rightarrow X = 20$ und $Y = 20$

- S_2 ist nicht ergebnisäquivalent zu einer seriellen Ausführung von T_3 , T_4
- Aber der finale Datenbankzustand ist konsistent – obwohl es einige Lost Updates gibt.

Übersicht

2 Schedules und Serialisierbarkeit

- Schedules
- Conflict Serializability
- Conflict Graphs (Precedence Graphs)
- Recoverable Schedules und Cascadeless Schedules

Eine vierte¹ Definition von Korrektheit: Conflict Serializability (Konfliktserialisierbarkeit)

Definition (D4¹)

Ein Schedule ist **conflict serializable** (konfliktserialisierbar) wenn er **konfliktäquivalent** zu einem seriellen Schedule ist.

¹Die dritte Definition (D3) ist Sichtenserialisierbarkeit.

Mögliche Konflikte zwischen Transaktionen

„Konflikte“ zwischen Paaren von Transaktionen (T_1 und T_2) und deren Operationen.

schedule S_A	
T_1	T_2
write(X, x)	read(X, x)

Konflikt

schedule S_B	
T_1	T_2
write(X, x)	write(X, x)

Konflikt

schedule S_C	
T_1	T_2
write(X, x)	read(X, x)

Konflikt

schedule S_D	
T_1	T_2
read(X, x)	read(X, x)

Kein Konflikt

Eine vierte Definition von Korrektheit: Conflict Serializability (Konfliktserialisierbarkeit)

Definition (D4)

Ein Schedule ist **conflict serializable** wenn er **konfliktäquivalent** zu einem seriellen Schedule ist.

- Seien I und J aufeinanderfolgende Operationen eines Schedules S von mehreren Transaktionen.
- Wenn I und J keinen Konflikt haben, können wir deren Reihenfolge vertauschen, um einen neuen Schedule S' zu erzeugen.
- Die Operationsreihenfolge ist in S und S' gleich, außer I und J.
- S und S' bezeichnen wir als **konfliktäquivalente Schedules**.

Menti

Fragen 12–15

Übersicht

2 Schedules und Serialisierbarkeit

- Schedules
- Conflict Serializability
- Conflict Graphs (Precedence Graphs)
- Recoverable Schedules und Cascadeless Schedules

Conflict Graph (Konfliktgraph)

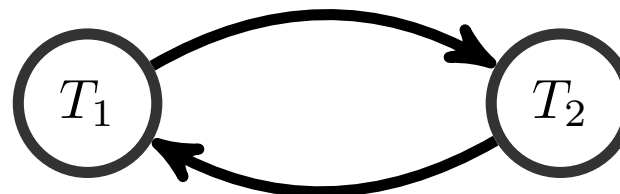
Erzeugen eines gerichteten Graphen (Conflict Graph oder Precedence Graph) für einen Schedule

Annahme: Eine Transaktion führt auf ein Datenobjekt immer ein Read aus, bevor auf das gleiche Objekt ein Write ausgeführt wird.

Gegeben ist ein Schedule für die Transaktionen $T_1, T_2, \dots, T_n$

- Die Knoten des Conflict Graphs sind die Transaktions-Ids.
- Eine Kante von T_i nach T_j zeigt einen Konflikt zwischen T_i und T_j an, wobei T_i den relevanten Zugriff früher durchführt.
- Manchmal werden Kantenlabels mit dem Namen des involvierten Datenobjekts annotiert.

Beispiel für einen Conflict Graph des Schedules S_1



Serialisierbarkeit feststellen

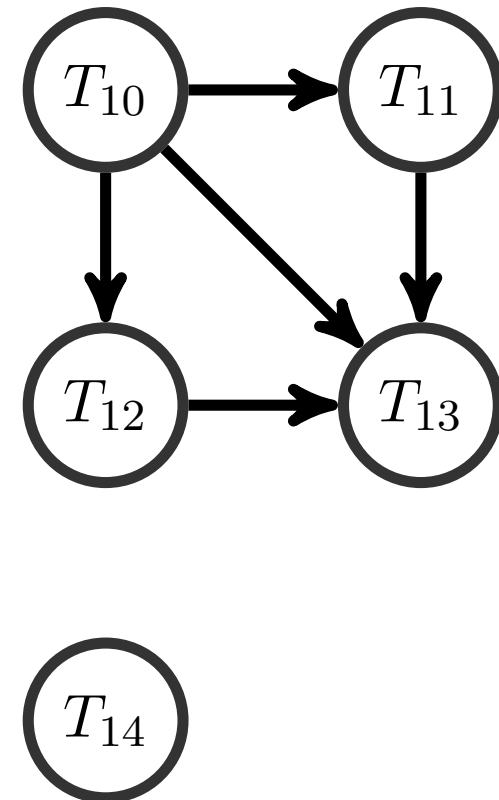
Gegeben ist ein Schedule S und ein Conflict Graph, we können wir feststellen, ob der Schedule conflict serializable ist?

- Ein Schedule ist **conflict serializable**, wenn der Conflict Graph **acyklisch** ist.
- Intuitiv: Ein Konflikt zwischen zwei Transaktionen erzwingt eine bestimmte Ausführungsreihenfolge (topologische Sortierung)

Wir verwenden conflict serializability (und keine andere Definition der Serialisierbarkeit), weil es eine praktikable Implementierung gibt.

Beispiel Conflict Graph

schedule S_6				
T_{10}	T_{11}	T_{12}	T_{13}	T_{14}
read(Y, y) read(Z, z)	read(X, x)			read(V, v) read(W, w) write(W, w)
	read(Y, y) write(Y, y)	read(Z, z) write(Z, z)		
read(T, t)			read(Y, y) write(Y, y) read(Z, z) write(Z, z)	
read(U, u)				



Welche der folgenden Schedules sind konfliktäquivalente serielle Schedules?

$T_{10}, T_{11}, T_{12}, T_{13},$ and T_{14} Yes

$T_{14}, T_{10}, T_{12}, T_{11},$ and T_{13} Yes

$T_{14}, T_{13}, T_{12}, T_{11},$ and T_{10} No

Übersicht

- 2 Schedules und Serialisierbarkeit
 - Schedules
 - Conflict Serializability
 - Conflict Graphs (Precedence Graphs)
 - Recoverable Schedules und Cascadeless Schedules

Transaktionen: Isolation und Atomicity

Transaktionen können fehlschlagen

Recoverable Schedules

schedule S_A	
T_i	T_j
read(X, x)	
write(X, x)	
	read(X, x)
	write(X, x)
	commit
rollback	

- Wenn T_i fehlschlägt, muss die Transaktion zurückgesetzt werden, um die **Atomicity (Atomarität)** Eigenschaft zu erhalten (siehe Recovery).
- Wenn eine andere Transaktion T_j Daten gelesen hat, die von T_i geschrieben wurden, dann muss auch T_j zurückgesetzt werden.
 $\Rightarrow$ DBS muss sicherstellen, dass Schedules recoverable sind.
- Dieser Schedule ist nicht recoverable.

Menti

Fragen 17-18

Recoverable (Wiederherstellbare) Schedules

Ein Schedule ist **recoverable**, wenn für jedes Transaktionspaar T_i und T_j gilt: Wenn T_j Daten liest die von T_i beschrieben wurden, dann muss T_i vor T_j committet werden.

schedule S_A	
T_i	T_j
read(X, x)	read(X, x) write(X, x) commit
write(X, x)	
rollback	

recoverable

schedule S_B	
T_i	T_j
read(Y, y)	read(X, x) write(X, x) commit
write(Y, y)	
rollback	

recoverable

Cascading Rollbacks (Kaskadierendes Rücksetzen)

schedule S_{11}		
T_{22}	T_{23}	T_{24}
read(A, a) read(B, b) write(A, a) write(B, b)	read(A, a)	read(A, a) read(B, b)
rollback		

- T_{22} Rollback $\Rightarrow$ wir müssen auch T_{23} und T_{24} zurücksetzen, weil diese "dirty data" lesen. (Cascading Rollback)
- Der Schedule ist nicht cascadeless (kommt nicht ohne kaskadierendes Rücksetzen aus).
- Aber der Schedule ist recoverable.

Menti

Frage 19

Cascadeless Schedules

Ein Schedule ist **cascadeless**, wenn für jedes Transaktionspaar T_i und T_j gilt: Wenn T_j Daten liest, die von T_i geschrieben wurden, dann muss T_i vor dem Lesezugriff von T_j bereits committed sein.

schedule S_{11}'		
T_{22}	T_{23}	T_{24}
read(A, a) read(B, b) write(A, a) write(B, b) rollback	read(A, a) commit	read(A, a) read(B, b) commit

Dieser Schedule ist auch recoverable

Cascading Rollbacks können verhindert werden, indem nur von Transaktionen gelesen wird, die bereits committed sind.

Cascadeless Schedules

- Jeder cascadeless Schedule ist auch recoverable.
- Cascading Rollbacks können schnell zeitaufwändig werden.
- Es ist sinnvoll, sich auf Schedules zu beschränken, die cascadeless sind.

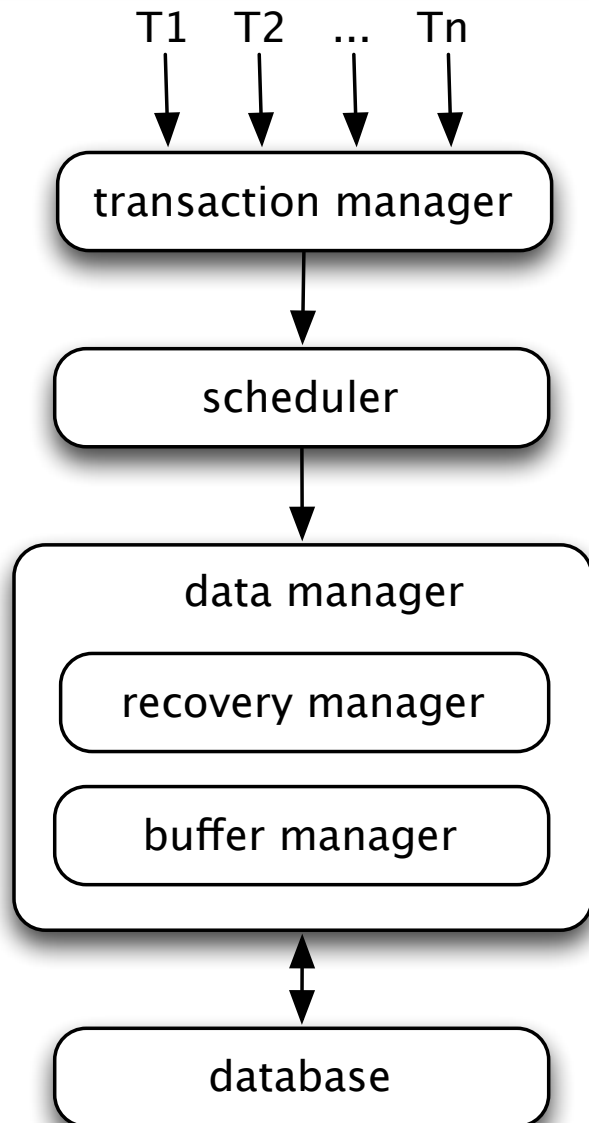
Zusammenfassung: Transaktionen und Schedules

- Jede Transaktion erhält den konsistenten Zustand der Datenbank.
- Die serielle Ausführung einer Menge von Transaktionen erhält einen konsistenten Zustand.
- Bei einer nebenläufigen Ausführung, können die einzelnen Ausführungsschritte einer Menge an Transaktionen verzahnt sein.
- Ein nebenläufiger Schedule ist serialisierbar, wenn dieser äquivalent zu einem seriellen Schedule ist.
 - Conflict Serializability
Bevorzugte Methode, da es praktikable Implementierungsmöglichkeiten gibt:
Conflict Graphs
- Schedules müssen recoverable und sollten cascadeless sein.

Übersicht I

- 1 Transaktionen
 - Eigenschaften
 - Operationen auf Transaktionsebene
 - Umsetzung der ACID-Eigenschaften
- 2 Schedules und Serialisierbarkeit
 - Schedules
 - Conflict Serializability
 - Conflict Graphs (Precedence Graphs)
 - Recoverable Schedules und Cascadeless Schedules
- 3 Mehrbenutzersynchronisation
 - Sperrbasierte Synchronisation
 - Zwei-Phasen-Sperrprotokoll (2PL)
 - Konvertierung von Sperren
 - Erkennung von Deadlocks

Der Datenbank-Scheduler



Aufgabe des Schedulers:
Operationen der Transaktionen $T_1, \dots, T_n$
in eine Reihenfolge bringen, die
serialisierbar ist (und ohne
kaskadierendes Rücksetzen auskommt).

Synchronisationsverfahren

- Pessimistisch
 - **Sperrbasierte Synchronisation**
 - Zeitstempel-basierte Synchronisation
- Optimistisch

Basierend auf "Datenbanksysteme: Ein Einführung"
by Alfons Kemper and Andre Eickler, Oldenbourg Verlag 2011.

Sperrbasierte Synchronisation

Sicherstellen von (konflikt-)serialisierbaren Schedules durch **Verzögern** von Transaktionen, die die Serialisierbarkeit verletzen würden.

Zwei Arten von Sperren können auf ein Datenobjekt Q angewendet werden

- S (shared lock, read lock, Lesesperre)
- X (exclusive lock, write lock, Schreibsperre)

Sperroperationen

- **lock_S(Q)** – setzt ein Shared Lock auf Datenobjekt Q
- **lock_X(Q)** – setzt ein Exclusive Lock auf Datenobjekt Q
- **unlock(Q)** – Freigabe des Locks auf Datenobjekt Q

Sperrbasierte Synchronisation

Privilegien bei Locks

Eine Transaktion, die

- ein Exclusive Lock hält, darf einen schreibenden oder lesenden Zugriff durchführen.
- ein Shared Lock hält, darf einen lesenden Zugriff durchführen.

Verträglichkeitsmatrix (Kompatibilitätsmatrix)

	NL	S	X
S	OK	OK	-
X	OK	-	-

NL – No Lock (keine Sperre)

- Nebenläufige Transaktionen dürfen nur kompatible Locks verwenden.
- Eine Transaktion muss ggf. auf Freigabe eines Locks warten.

Ein Beispiel

- T_{15} überweist 50 EUR von Konto B auf Konto A.
- T_{16} zeigt den gesamten Kontostand der Konten A und B.

T_{15}	T_{16}
lock_X(B) read(B, b) $b \leftarrow b - 50$ write(B, b) unlock(B) lock_X(A) read(A, a) $a \leftarrow a + 50$ write(A, a) unlock(A)	lock_S(A) read(A, a) unlock(A) lock_S(B) read(B, b) unlock(B) display(A+B)

Resultat bei serieller Ausführung:

T_{16} zeigt 300

Bei diesem Schedule:

T_{16} zeigt 250

Ursache des Problems:

Der Exclusive Lock auf B wurde zu früh freigegeben.

Initial: $A = 100$ und $B = 200$

schedule S_7	
T_{15}	T_{16}
lock_X(B) read(B, b) $b \leftarrow b - 50$ write(B, b) unlock(B)	lock_S(A) read(A, a) unlock(A) lock_S(B) read(B, b) unlock(B) display(A+B)
lock_X(A) read(A, a) $a \leftarrow a + 50$ write(A, a) unlock(A)	

Probleme bei zu früher Freigabe

- Initial: $A = 100$ und $B = 200$
- Serieller Schedule $T_{15};T_{16}$ zeigt 300
- Serieller Schedule $T_{16};T_{15}$ zeigt 300
- S_7 zeigt 250

Frühe Sperrfreigaben können zu **inkorrekten** Resultaten führen (Non-Serializable Schedules), aber sie erlauben einen höheren Grad an Nebenläufigkeit.

Outline

- 3 Mehrbenutzersynchronisation
 - Sperrbasierte Synchronisation
 - Zwei-Phasen-Sperrprotokoll (2PL)
 - Konvertierung von Sperren
 - Erkennung von Deadlocks
 - Vermeidung von Deadlocks

Menti

Fragen 6–9

2PL: Ja oder Nein?

schedule S_A	schedule S_B		schedule S_C		schedule S_D
T_1	T_2	T_3	T_4	T_5	T_6
lock_X(A)	lock_X(A)		lock_X(A)		lock_X(A)
lock_X(B)	lock_X(B)			lock_X(B)	lock_X(B)
lock_X(C)	lock_X(C)			lock_X(C)	unlock(B)
unlock(A)	unlock(B)			unlock(C)	lock_X(C)
unlock(C)		lock_X(B)		unlock(B)	unlock(A)
unlock(B)	unlock(C)		unlock(A)		unlock(C)
Ja	unlock(A)	unlock(B)	Ja		Nein
	Ja				

Eigenschaften des Zwei-Phasen-Sperrprotokolls

- 2PL erzeugt nur serializable Schedules
 - Garantiert Konfliktserialisierbarkeit
 - 2PL erzeugt eine Menge aller möglichen serializable Schedules
- 2PL schützt nicht vor Deadlocks
- 2PL schützt nicht for Cascading Rollbacks
 - “Dirty” Reads sind möglich (Lesen von Transaktionen die noch nicht committed wurden)

Cascading Rollbacks

Ein Abort einer Transaktion kann zu einem Abort anderer Transaktionen führen.

schedule S_{11}			schedule S_{11}'		
T_{22}	T_{23}	T_{24}	$T_{22'}$	$T_{23'}$	$T_{24'}$
lock_X(A) lock_X(B) unlock(A) abort	 lock_X(A) unlock(A)	 lock_X(A)	lock_X(A) lock_X(B) unlock(A) commit	 lock_X(A) unlock(A) commit	 lock_X(A)

- Diese Schedules verwenden 2PL.
- Abort in $T_{22} \Rightarrow T_{23}$ und T_{24} müssen ebenfalls ein Abort auslösen.

Wie könne wir diese Cascading Rollbacks verhindern?

Transaktionen dürfen keine uncommitten Daten lesen (siehe S_{11}').

Striktes und rigoroses Zwei-Phasen-Sperrprotokoll

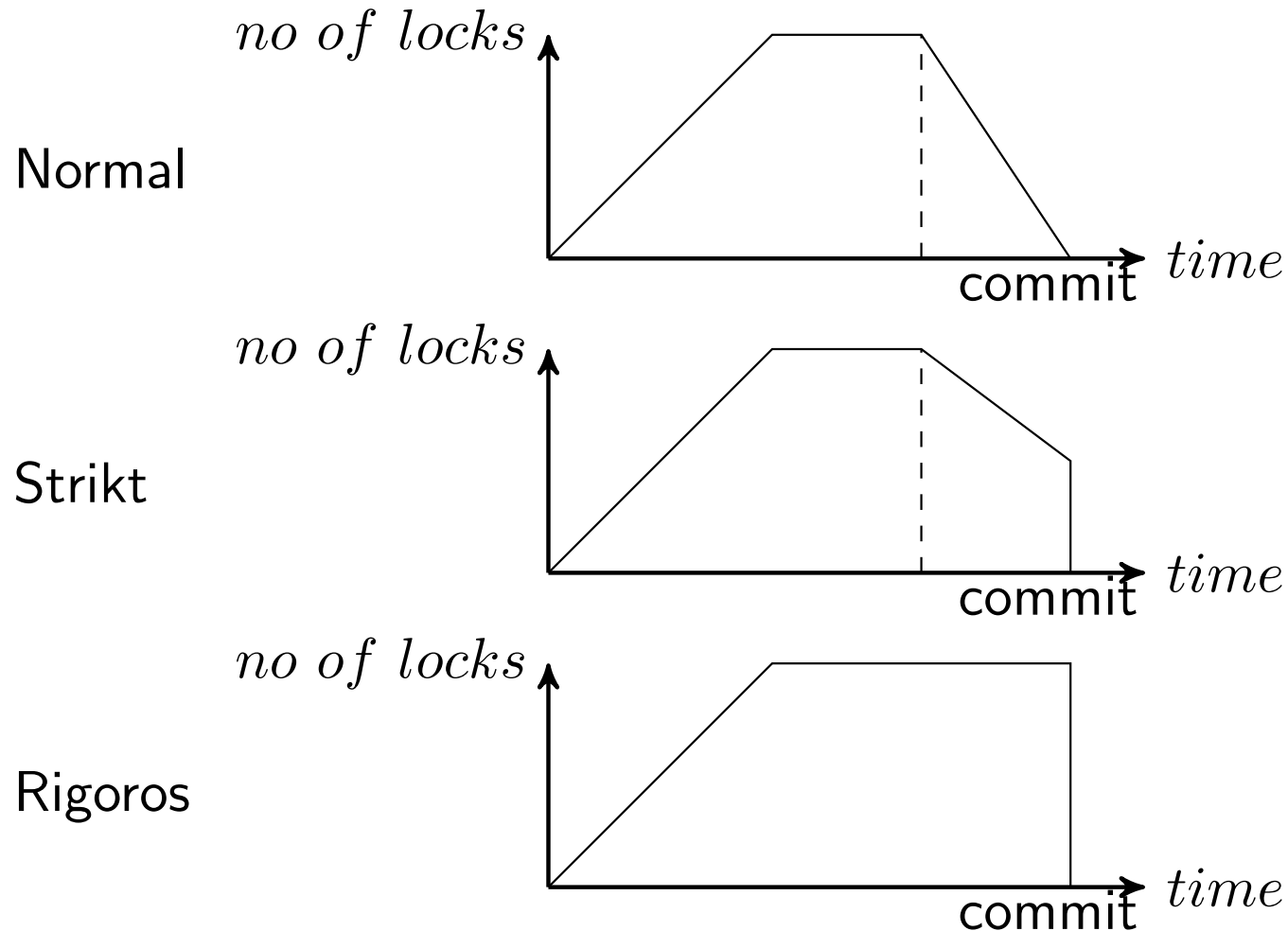
Striktes 2PL

- **Exclusive** Locks werden nicht vor dem Commit freigegeben.
- Verhindert “Dirty Reads”

Rigoroses 2PL:

- **Alle** Locks werden erst nach dem Commit freigegeben.
- Transaktionen können in der Commit-Reihenfolge serialisiert werden.
- Vorteil:
Keine Cascading Rollbacks
- Nachteil:
Weniger Nebenläufigkeit

Übersicht: 2PL Protokolle



Rigorous 2PL: Transaktionen können in Commit-Reihenfolge serialisiert werden.

Menti

Fragen 10–12

Normales, striktes oder rigoroses 2PL?

schedule S_1
T_1

lock_S(A)
lock_S(B)
lock_X(B)
lock_S(C)
unlock(A)
unlock(C)
 commit

Strikt

schedule S_2
T_2

lock_S(A)
lock_S(B)
lock_X(B)
 commit

Rigoros

schedule S_3
T_3

lock_S(A)
lock_S(B)
lock_X(B)
unlock(B)
lock_S(C)
unlock(A)
 commit

keine 2 Phasen

Übersicht

- 3 Mehrbenutzersynchronisation
 - Sperrbasierte Synchronisation
 - Zwei-Phasen-Sperrprotokoll (2PL)
 - Konvertierung von Sperren
 - Erkennung von Deadlocks
 - Vermeidung von Deadlocks

Deadlocks

2PL kann Deadlocks nicht verhindern.

T_1	T_2	
lock_X(A)	lock_S(B) read(B)	
read(A) write(A)		
lock_X(B)	lock_S(A)	T_1 muss warten auf T_2 T_2 muss warten auf T_1 $\Rightarrow$ Deadlock
...	...	

Lösungen

- **Erkennung von Deadlocks und anschließend Recovery**
- **Prävention**
- Timeout

Erkennung von Deadlocks

Erstellen eines Wartegraphen (“Wait-for graph”) und Prüfung auf Zyklen

- Ein Knoten für jede aktive Transaktion T_i
- Kante $T_i \rightarrow T_j$ wenn T_i auf die Lock-Freigabe von T_j wartet

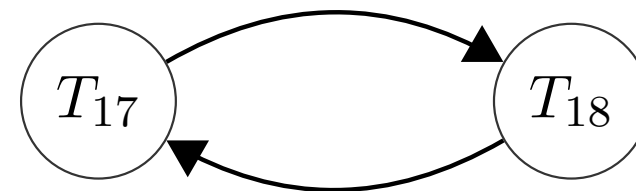
Ein Deadlock existiert, wenn der Wartegraph einen Zyklus hat.

Erkennung von Deadlocks

Wenn ein Deadlock erkannt wird:

- Ein passendes Opfer auswählen
- Abbruch des Opfers und Freigabe aller zugehörigen Sperren

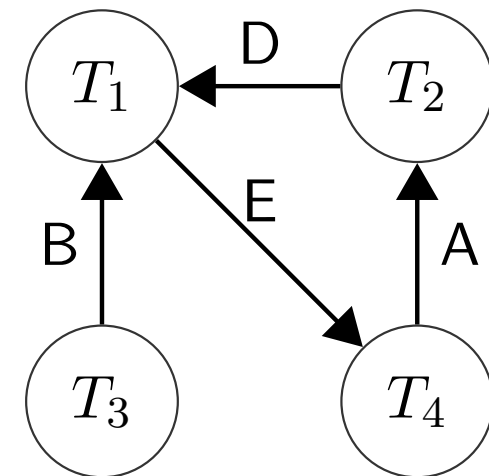
schedule S_8	
T_{17}	T_{18}
lock_X(B) read(B, b) $b \leftarrow b - 50$ write(B, b)	lock_S(A) read(A, a) lock_S(B)
lock_X(A)	



Erkennung von Deadlocks - Ein Beispiel

schedule S_A

T_1	T_2	T_3	T_4
	lock_X(A)		
lock_X(B) lock_X(C)		lock_X(B)	
lock_X(D)			lock_X(E) lock_X(A)
lock_X(E)	lock_X(D)		



Zyklus zwischen T_1 , T_4 und T_2
 $\Rightarrow$ Deadlock erkannt

Rollback von einer oder mehreren involvierten
 Transaktionen um den Deadlock zu lösen

Menti

Frage 13

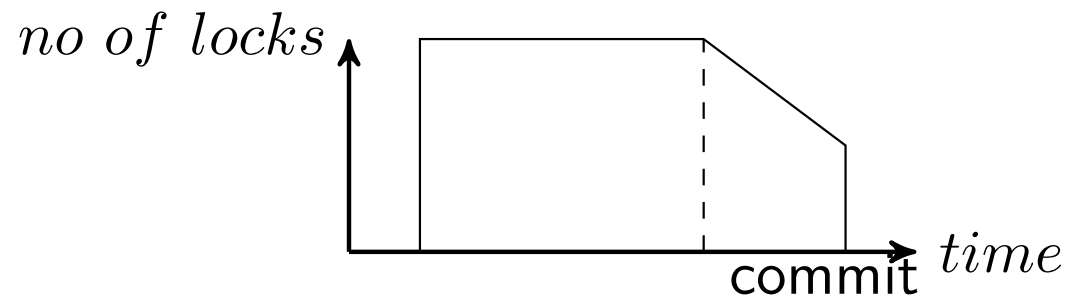
Übersicht

- 3 Mehrbenutzersynchronisation
 - Sperrbasierte Synchronisation
 - Zwei-Phasen-Sperrprotokoll (2PL)
 - Konvertierung von Sperren
 - Erkennung von Deadlocks
 - Vermeidung von Deadlocks

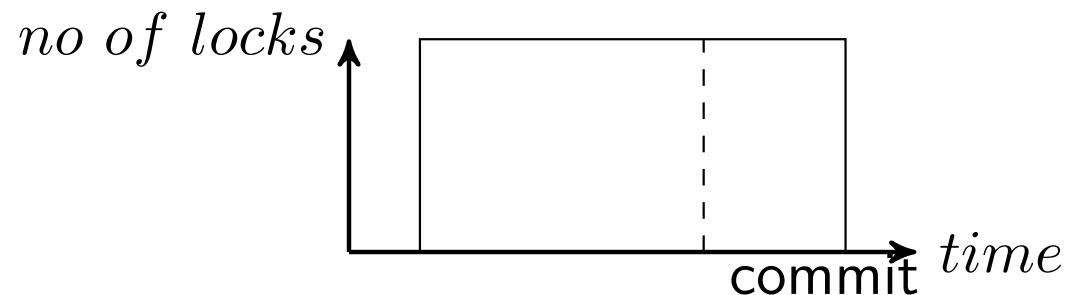
Konservatives 2PL-Protokoll

- Normales/Striktes/Rigoroses 2PL können Deadlocks nicht verhindern.
- Zusätzliche Anforderung:
Alle Locks (Shared und Exclusive) werden gleich zu Beginn einer Transaktion gesetzt.

konservatives striktes 2PL



konservatives rigoroses 2PL



Nur in wenigen Applikationen verwendbar

Zusammenfassung Mehrbenutzersynchronisation

- Mehrere Protokolle zur Mehrbenutzersynchronisation wurden entwickelt
 - Hauptziel: nur serialisierbare, recoverable und cascadeless Schedules zulassen
 - Zwei-Phasen-Sperrprotokoll
 - Die meisten relationalen DBMS benutzen rigoroses 2PL
- Erkennung von Deadlocks (Wartegraph) und Verhindern von Deadlocks (konservatives 2PL)
- Serialisierbarkeit vs. Nebenläufigkeit

Übersicht

- 4 Recovery
 - Fehlerklassifikation
 - Datenspeicher
 - Log-Einträge
 - Log-Based Recovery

Recovery

Aspekte

- Atomarität
 - Transaktionen können ein Abort ausführen (Rollback).
- Dauerhaftigkeit
 - Was muss passieren, wenn das DBMS abstürzt?

Das DBMS garantiert, dass eine Transaktion

- entweder fertig wird und ein permanentes Resultat liefert (committed)
- oder keinen Effekt auf die Datenbank hat (aborted).

Die Rolle der **Recovery**-Komponente ist es, die Atomarität und Dauerhaftigkeit von Transaktionen trotz etwaiger Systemfehler zu garantieren.

Wie kann Dauerhaftigkeit garantiert werden?

- Eine Transaktion verändert Daten im Arbeitsspeicher.
- Die Daten wurden **noch nicht** auf die Festplatte geschrieben.
- Commit wird durchgeführt.

Der Benutzer nimmt an, die Transaktion wäre erfolgreich abgeschlossen und alle Änderungen sind in der Datenbank gespeichert.

- Was passiert bei einem Blackout?
- Welche Daten sind in der Datenbank?

Wie kann Dauerhaftigkeit garantiert werden?

- Eine Transaktion verändert Daten im Arbeitsspeicher.
- Die Daten wurden **teilweise** auf die Festplatte geschrieben.
- Commit wird durchgeführt.

Der Benutzer nimmt an, die Transaktion wäre erfolgreich abgeschlossen und alle Änderungen sind in der Datenbank gespeichert.

- Was passiert bei einem Blackout?
- Welche Daten sind in der Datenbank?

Wie kann Dauerhaftigkeit garantiert werden?

- Eine Transaktion verändert Daten im Arbeitsspeicher.
- Die Daten wurden **vollständig** auf die Festplatte geschrieben.
- Commit wird durchgeführt.

Der Benutzer nimmt an, die Transaktion wäre erfolgreich abgeschlossen und alle Änderungen sind in der Datenbank gespeichert.

- Was passiert bei einem Hardware Fehler?
⇒ Verlust einer Festplatte
- Welche Daten sind in der Datenbank?

Wie kann Dauerhaftigkeit garantiert werden?

- Eine Transaktion verändert Daten im Arbeitsspeicher.
- Die Daten wurden **vollständig** auf **mehrere** Festplatten geschrieben.
- Commit wird durchgeführt.

Der Benutzer nimmt an, die Transaktion wäre erfolgreich abgeschlossen und alle Änderungen sind in der Datenbank gespeichert.

- Was passiert, wenn es Feuer, Flut, Erdbeben oder sonst etwas gibt?
⇒ alle Festplatten sind verloren
- Welche Daten sind in der Datenbank?

Wie kann Dauerhaftigkeit garantiert werden?

- Eine Transaktion verändert Daten im Arbeitsspeicher.
- Die Daten wurden **vollständig** auf **mehrere** Festplatten geschrieben und die Festplatten wurden auf **mehreren geographische verschiedenen** Computerzentren verteilt.
- Commit wird durchgeführt.

Der Benutzer nimmt an, die Transaktion wäre erfolgreich abgeschlossen und alle Änderungen sind in der Datenbank gespeichert.

- Was passiert wenn es Feuer, Flut, Erdbeben oder sonst etwas bei allen Computerzentren gibt?
⇒ alle Computerzentren sind verloren
- Welche Daten sind in der Datenbank?

Dauerhaftigkeit (Durability)

- Dauerhaftigkeit ist **relativ** und abhängig von der Anzahl der Kopien und den geographischen Orten.
- Garantien sind nur möglich, wenn
 - wir zuerst die Kopien aktualisieren und
 - erst dann den Benutzer informieren, dass das Commit der Transaktion erfolgreich war.

Wir nehmen deshalb an, dass die WAL-(Write Ahead Logging)-Regel erfüllt ist.

Variationen der WAL-Regel:

- **Log-Based Recovery**
- Volle Redundanz: Spiegeln aller Daten auf mehreren Computern (Festplatten, Rechenzentren), die alle das gleiche machen

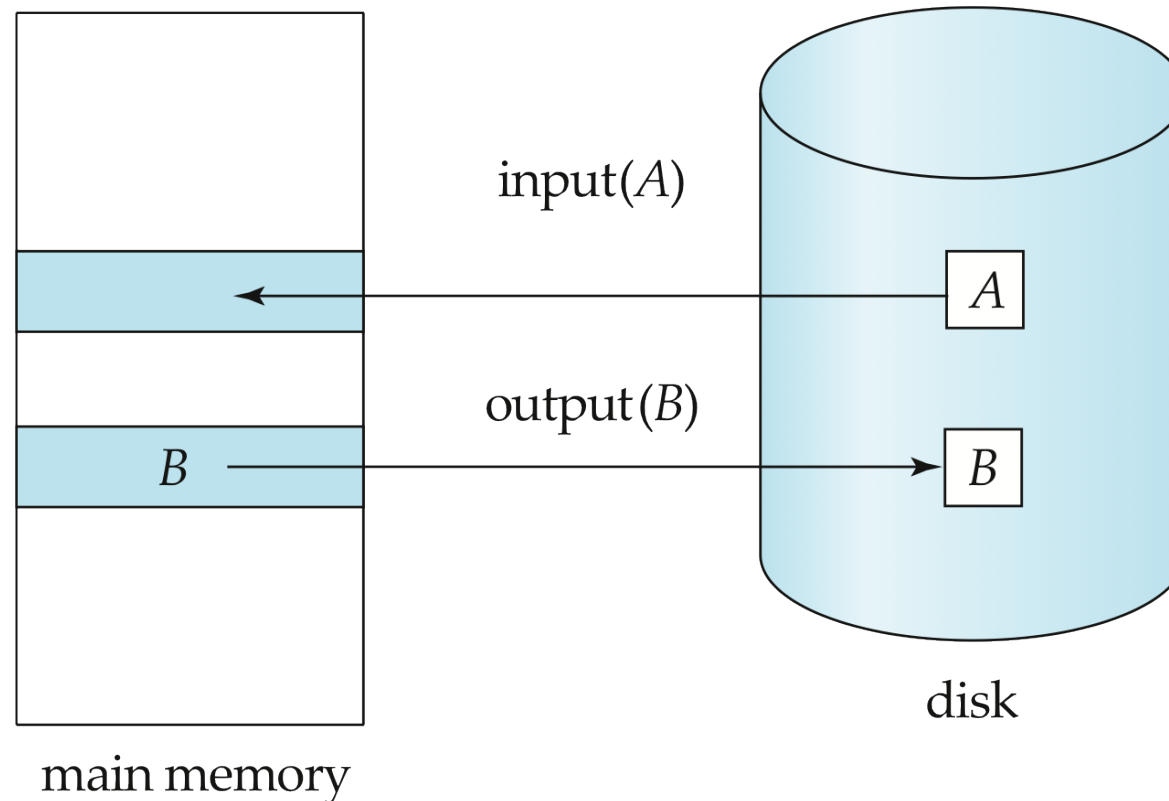
Übersicht

4 Recovery

- Fehlerklassifikation
- **Datenspeicher**
- Log-Einträge
- Log-Based Recovery

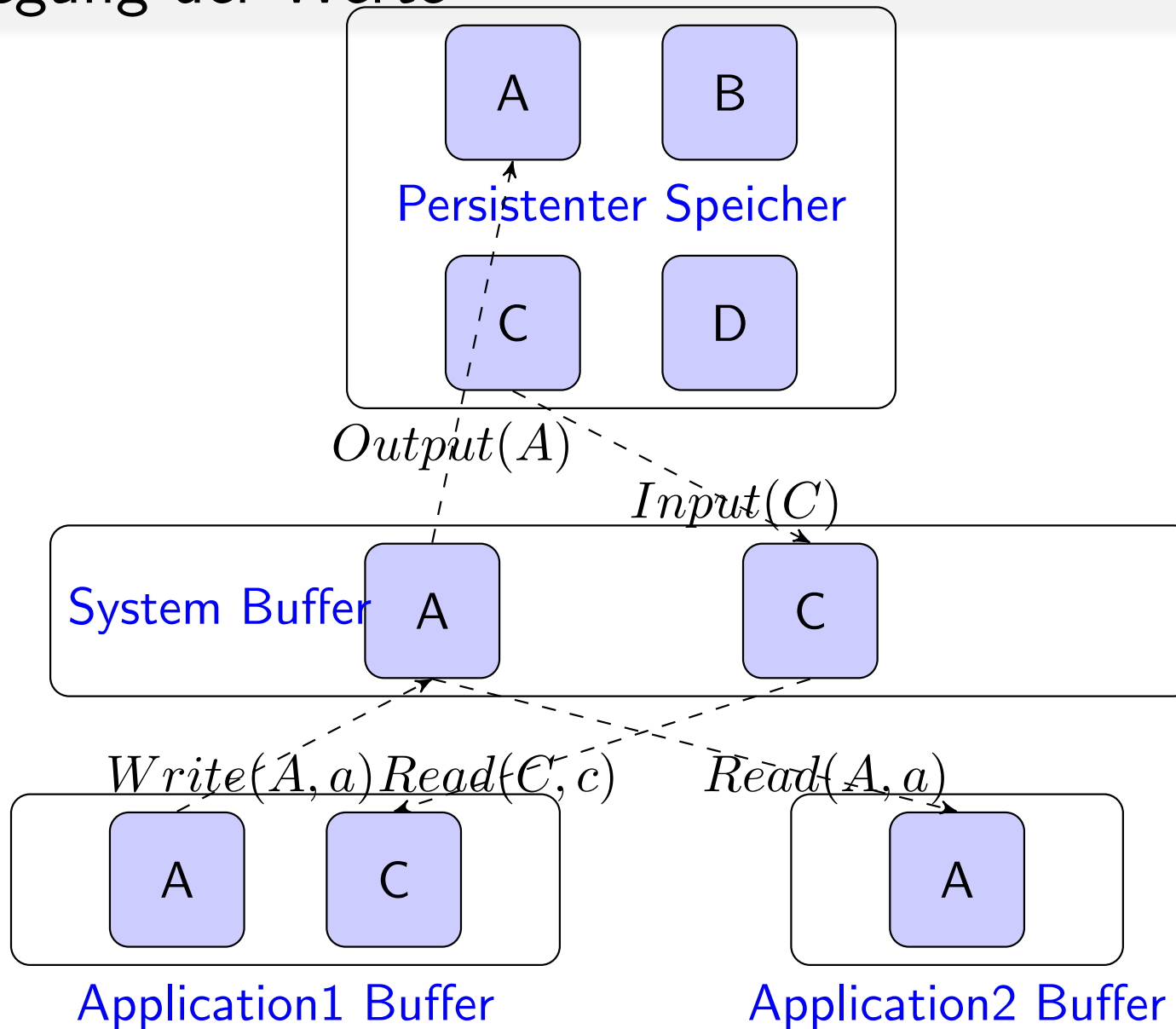
Zweistufige Speicherhierarchie

Daten werden in Seiten (Pages) und Blöcken (Blocks) organisiert.



- **Flüchtiger Speicher (Volatile Storage)** (Arbeitsspeicher)
- **Nichtflüchtiger Speicher (Non-Volatile Storage)** (Fesplatte)
- Stabiler Speicher (Stable storage) (RAIDS, Remote Backups, ...)

Bewegung der Werte



Übersicht

4 Recovery

- Fehlerklassifikation
- Datenspeicher
- Log-Einträge
- Log-Based Recovery

Die WAL-Regel für Log-Based Recovery

WAL (Write Ahead Logging)

- Bevor eine Transaktion in den **Commit**-Zustand wechselt, müssen „alle zugehörigen“ Log-Einträge auf einen stabilen Speicher geschrieben worden sein, inkl. dem Commit-Log-Eintrag.
- Bevor eine modifizierte Seite (oder Block) im Arbeitsspeicher in die Datenbank (Nichtflüchtiger Speicher) geschrieben werden kann, müssen „alle zugehörigen“ Log-Einträge auf einen stabilen Speicher geschrieben worden sein.

Logging

Im normalen Betrieb

- Am Beginn registriert eine Transaktion T sich selbst im **Log**: $[T \text{ start}]$
- Wenn ein Datenobjekt X bearbeitet wird durch $\text{write}(X, x)$

① Folgender Log-Eintrag wird hinzugefügt

- $[T, X, V\text{-alt}, V\text{-neu}]$
- Transaktions-ID (d.h., T)
- Name des Datenobjekts (d.h., X)
- Alter Wert des Objekts
- Neuer Wert des Objekts

② Schreibe den neuen Wert von X

Der Buffer Manager schreibt die neuen Werte später asynchron auf die Festplatte.

- Am Ende der Transaktion fügt Transaktion T den Eintrag $[T \text{ commit}]$ ins Log ein

Eine Transaktion gilt als committed genau dann, wenn der Commit-Eintrag (nach allen vorherigen Einträgen) im Log steht!

Struktur eines Log-Eintrags (Log Record)

[TID, DID, old, new]

TID ID der Transaktion, die die Änderung verursacht hat

DID ID des Datenobjekts

Ort auf der Festplatte (Page, Block, Offset)

old Wert des Datenobjekts vor der Änderung

new Wert des Datenobjekts nach der Änderung

Zusätzliche Einträge

start Transaktion TID wurde gestartet [TID start]

commit Transaktion TID wurde committed [TID commit]

abort Transaktion TID wurde aborted [TID abort]

Menti

Frage 14–17

Was stimmt hier nicht?

[T1 commit]

[T1, B, 300, 400]

[T1 start]

Commit vor Start

[T1 start]

[T1, B, 300, 400]

[T1 commit]

[T1, C, 40, 540]

Es darf keine Log-Einträge für T1 nach dem Commit geben.

Übersicht

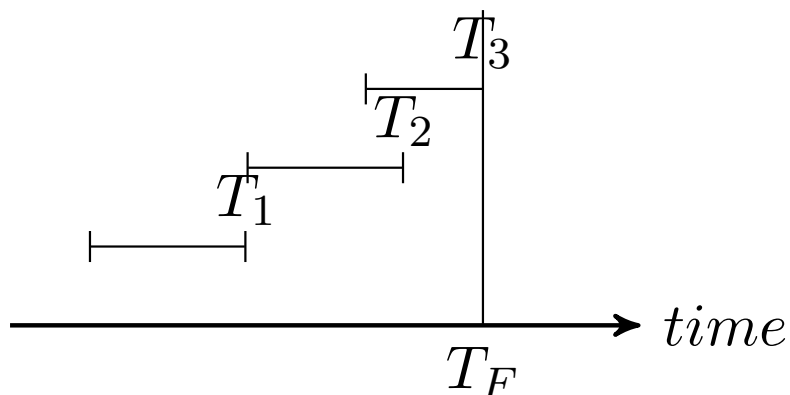
- 4 Recovery
 - Fehlerklassifikation
 - Datenspeicher
 - Log-Einträge
 - Log-Based Recovery

Log-Based Recovery

Operationen zur Wiederherstellung nach Fehlern

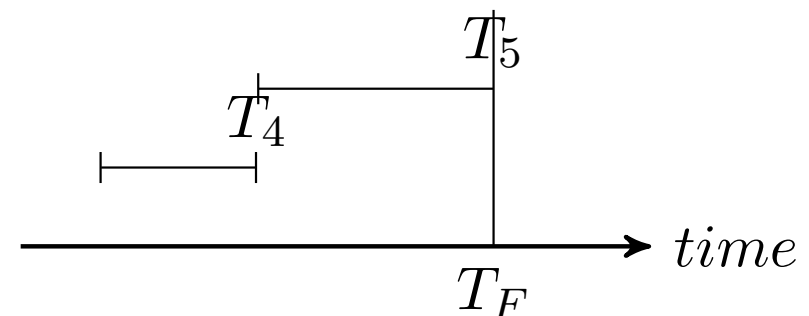
- **Redo**: Die Änderungen an der Datenbank wiederholen
- **Undo**: Den Datenbankzustand vor der Ausführung wiederherstellen

Systemfehler



- Redo T_1 and T_2
- Undo T_3

Systemfehler



- Redo T_4
- Undo T_5

Recovery-Algorithmus

Um die Datenbank nach einem Fehler wiederherzustellen

- Reproduzieren (redo) der Resultate von Transaktionen, die committed wurden
- Rückgängigmachen (undo) von Transaktionen, die noch nicht committed wurden

Anmerkungen

- In einem Multitasking-System, müssen eventuell mehr als eine Transaktion rückgängig gemacht werden.
- Wenn ein Systemabsturz während der Recovery-Phase auftritt, muss auch ein erneuter Recovery-Anlauf korrekte Resultate liefern (**Idempotenz**).

Log-Based Recovery

Datenbank

A	100
B	300
C	5
D	60
E	80

Log-Einträge

[T1 start]
[T1, B, 300, 400]
[T1, C, 5, 10]
[T2 start]
[T2, E, 80, 480]
[T1, A, 100, 560]
[T1 commit]
[T2, A, 560, 570]
[T2, D, 60, 530]

Wie würden Sie die Log-Informationen verwenden (systematisch), um die Datenbank nach einem Absturz wiederherzustellen?

Drei Recovery-Phasen

- ① Redo (Vollständige Wiederholung der Historie)
 - Alle Log-Einträge Schritt für Schritt durchgehen
 - **Alle** Änderungen in derselben Reihenfolge wie im Log anwenden
 - Bestimmen der „Undo“ Transaktionen
 - $[T_i \text{ start}]$: T_i zur „Undo List“ hinzufügen
 - $[T_i \text{ abort}]$ oder $[T_i \text{ commit}]$: T_i von der „Undo List“ entfernen
- ② Undo (Rollback) aller Transaktionen in der „Undo List“
 - Alle Log-Einträge rückwärts durchgehen
 - Rückgängigmachen aller Änderungen von Transaktionen in der „Undo List“ und Erstellung eines Compensation-Log-Eintrags
 - Für jeden $[T_i \text{ start}]$ Eintrag einer Transaktion T_i in der „Undo List“, füge einen $[T_i \text{ abort}]$ Eintrag ins Log ein, entferne T_i von der „Undo List“
 - Stopp, sobald die „Undo List“ leer ist

Compensation-Log-Einträge

[TID, DID, value]

- Erstellt zum Rückgängigmachen (Ausgleichen/Kompensieren) der Änderungen von [TID, DID, value, newValue]
- Redo-Only-Log-Eintrag
- Kann auch für einen Rollback einer Transaktion während der normalen Ausführung verwendet werden.

Beispiel

Phase 2 (Undo)

Datenbank

A	560
B	400
C	10
D	60
E	80

Undo List

{ }

Log-Einträge

[T1 start]

[T1, B, 300, 400]

[T1, C, 5, 10]

[T2 start]

[T2, E, 80, 480]

[T1, A, 100, 560]

[T1 commit]

[T2, A, 560, 570]

[T2, D, 60, 530]

[T2, D, 60]

[T2, A, 560]

[T2, E, 80]

[T2 abort]

ARIES

- State-Of-The-Art Methode
- Erweitert den vorgestellten Algorithmus um einige Optimierungen und einer Vorab-Phase: Durchgehen des Logs um Dirty-Pages zu identifizieren und um den „Startpunkt“ des Logs sowie die „Undo“ Transaktionen zu bestimmen
- Behandlung von Dirty Pages:
DirtyPageTable, Erweiterung der Log-Einträge (pageID), Erweiterung der Pages (pageLSN, letzter Log-Eintrag der Änderungen vorgenommen hat),...

Log-Einträge in ARIES haben eine Log Sequence Number (LSN):
[LSN, TransactionID, PageID, redoValue, undoValue, prevLSN]

Compensation Log-Eintrag in ARIES (CLR):
[LSN, TransactionID, PageID, redoValue, prevLSN, undoNxtLSN]

Zusammenfassung Recovery

- Ziel ist das Sicherstellen von Atomarität und Dauerhaftigkeit trotz Systemfehlern und Abstürzen
- Dauerhaftigkeit ist relativ
- WAL-Regel
- Log-Based-Recovery
 - Alle Änderungen müssen in eine Log-Datei geschrieben werden
 - Eine Transaktion führt ein Commit genau dann durch, wenn der Commit-Eintrag im Log geschrieben wurde.

- 5 Schedules and serializability
 - Serializable schedules
 - Conflict serializability
 - Conflict graphs (precedence graphs)

- 6 Concurrency
 - Deadlock detection

- 7 Recovery
 - Log-based recovery
 - Checkpoints
 - No-undo/no-redo algorithm

View equivalent schedules

Two schedules S_1 and S_2 are **view equivalent** ($S_1 \equiv S_2$) if

- ① The set of transactions participating in S_1 and S_2 are the same.
- ② If T_i in S_1 reads data item Q and Q was written by T_j , then the same must hold in S_2
 - The same values are read by all transactions in both S_1 and S_2
 $\Rightarrow$ the same computation occurs
 - Note that this also applies to the first read of Q in a transaction
- ③ If T_i in S_1 executes the last write on Q , then the same holds in S_2 .
 - This ensures the same final system state for both schedules.

A third notion of correctness (view serializability)

Definition (D3)

Let $\{T_1, T_2, \dots, T_n\}$ be a set of transactions participating in schedule S . S is a **view serializable** schedule if there exists a serial schedule S_s such that $S \equiv S_s$.

Pros and Cons

- View serializability provides a convenient definition of correctness
- NP-complete problem to check if schedule is view serializable
- Needs a definition that can be checked in practice

A third notion of correctness (view serializability)

Definition (D3)

Let $\{T_1, T_2, \dots, T_n\}$ be a set of transactions participating in schedule S . S is a **view serializable** schedule if there exists a serial schedule S_s such that $S \equiv S_s$.

schedule S_B		schedule S_C		schedule S_D	
T_1	T_2	T_1	T_2	T_1	T_2
read(X, x)	write(X, x)	read(X, x)	read(X, x)	read(X, x)	read(X, x)
write(X, x)			write(X, x)	write(X, x)	
write(X, x)			write(X, x)		

Not view serializable

Not view serializable

View serializable

Explain View Serializable, Example One

Example

schedule S_B		schedule $S_{B_{one}}$		schedule $S_{B_{two}}$	
T_1	T_2	T_1	T_2	T_1	T_2
read(X, x)	write(X, x)	read(X, x)	write(X, x)	read(X, x) write(X, x) write(X, x)	write(X, x)
write(X, x)		write(X, x)			
write(X, x)		write(X, x)			

- Original schedule
- T_1 input X: first
- T_2 input: None
- Last write X: T_1

- Serial schedule one
- T_1 input X: first
- T_2 input: None
- Last write X: T_2

- Serial schedule two
- T_1 input X: T_2
- T_2 input: None
- Last write X: T_1

Explain View Serializable, Example Two

Example

schedule S_C		schedule $S_{C_{one}}$		schedule $S_{C_{two}}$	
T_1	T_2	T_1	T_2	T_1	T_2
read(X, x)	read(X, x)	read(X, x)			read(X, x)
	write(X, x)	write(X, x)	read(X, x)	read(X, x)	write(X, x)
write(X, x)			write(X, x)	write(X, x)	

- Original schedule
- T_1 input X: first
- T_2 input X: first
- Last write X: T_1

- Serial schedule one
- T_1 input X: first
- T_2 input X: T_1
- Last write X: T_2

- Serial schedule two
- T_1 input X: T_2
- T_2 input X: first
- Last write X: T_1

Explain View Serializable, Example Three

Example

schedule S_D		schedule $S_{D_{one}}$		schedule $S_{D_{two}}$	
T_1	T_2	T_1	T_2	T_1	T_2
read(X, x)	read(X, x)	read(X, x)	read(X, x)	read(X, x)	read(X, x)
write(X, x)		write(X, x)		write(X, x)	

- Original schedule
- T_1 input X: first
- T_2 input X: first
- Last write X: T_1

- Serial schedule one
- T_1 input X: first
- T_2 input X: T_1
- Last write X: T_1

- Serial schedule two
- T_1 input X: first
- T_2 input X: first
- Last write X: T_1

5 Schedules and serializability

- Serializable schedules
- Conflict serializability
- Conflict graphs (precedence graphs)

6 Concurrency

- Deadlock detection

7 Recovery

- Log-based recovery
- Checkpoints
- No-undo/no-redo algorithm

View vs. Conflict Serializable

schedule S_5		
T_1	T_2	T_3
read(X, x)	write(X, x)	write(X, x)
write(X, x)		

- S_5 is view serializable, but is not conflict serializable, because every pair of consecutive instructions conflict.
- T_2 and T_3 have write instructions without read instructions, termed **blind writes**.
- Blind writes appear in any view serializable schedule that is not conflict serializable.

- 5 Schedules and serializability
 - Serializable schedules
 - Conflict serializability
 - Conflict graphs (precedence graphs)

- 6 Concurrency
 - Deadlock detection

- 7 Recovery
 - Log-based recovery
 - Checkpoints
 - No-undo/no-redo algorithm

5 Schedules and serializability

- Serializable schedules
- Conflict serializability
- Conflict graphs (precedence graphs)

6 Concurrency

- Deadlock detection

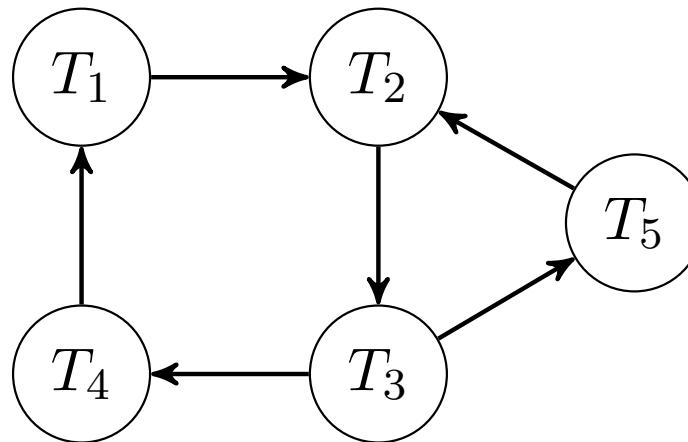
7 Recovery

- Log-based recovery
- Checkpoints
- No-undo/no-redo algorithm

Multiple cycles

Wait-for graph with multiple cycles

- $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow T_1$
- $T_2 \rightarrow T_3 \rightarrow T_5 \rightarrow T_2$

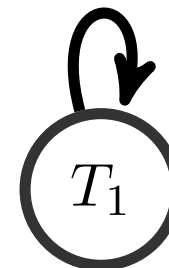
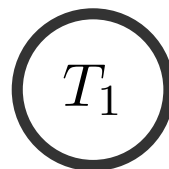


By rolling back T_3 , we can resolve both cycles

Wait-for graph

What is the wait-for graph for the schedule?

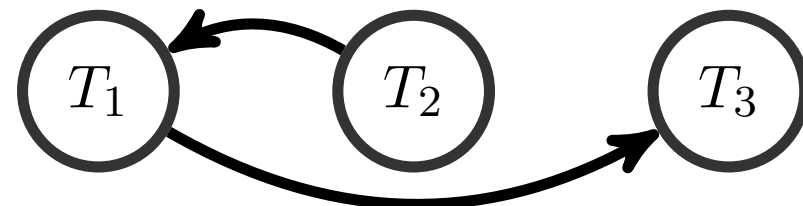
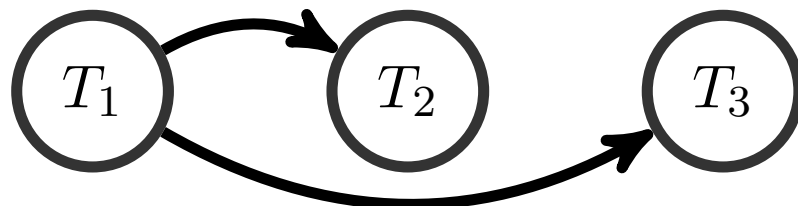
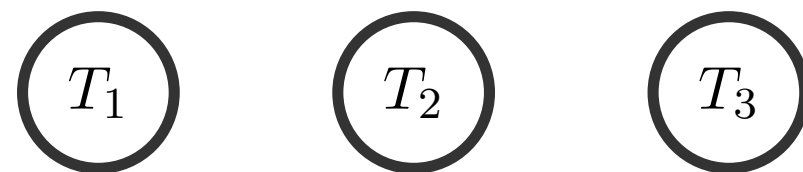
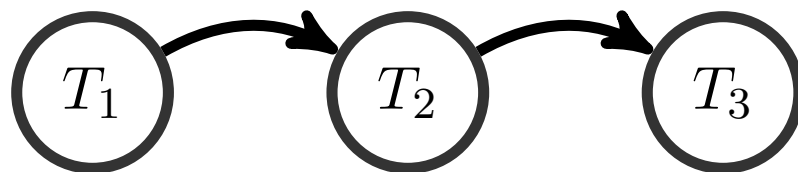
$$\frac{\frac{\text{schedule } S_A}{T_1}}{\text{lock_S(A)} \quad \text{lock_X(A)}}$$



Wait-for graph

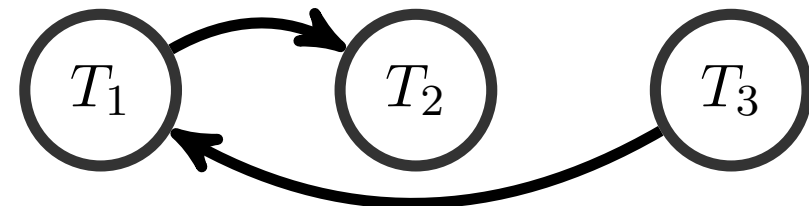
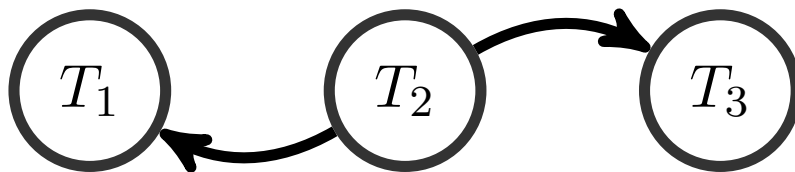
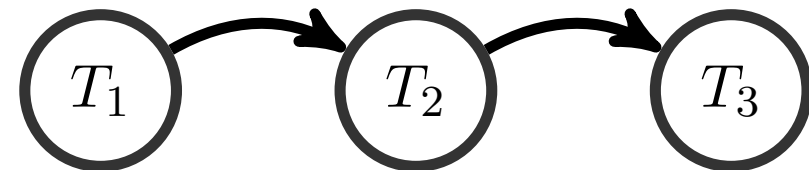
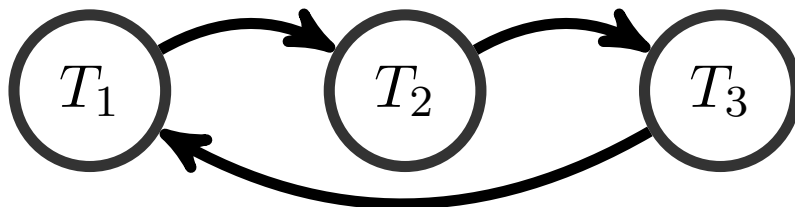
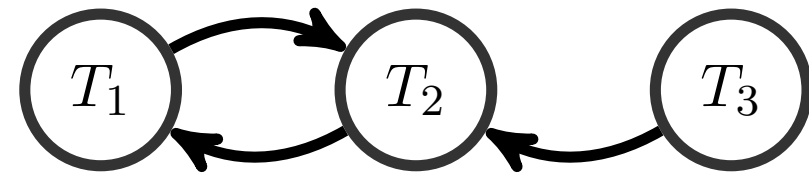
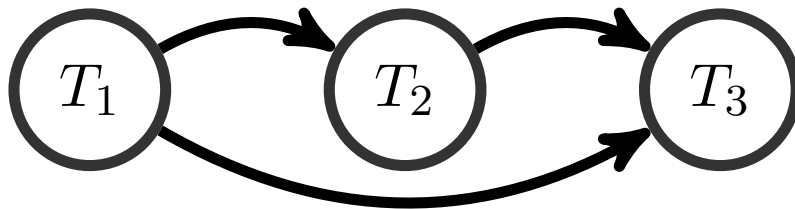
What is the wait-for graph for the schedule?

schedule S_A		
T_1	T_2	T_3
lock_X(A)	lock_X(B) lock_X(D)	lock_X(C)
lock_X(E)		



Wait-for graph

Which schedules have a deadlock?



Example log

Transactions T_1 and T_2 are executed **sequentially**, the initial values are:
A=100 B=300 C=5 D=60 E=80

T_1

read(A, a)

$a \leftarrow a + 50$

read(B, b)

$b \leftarrow b + 100$

write(B, b)

read(C, c)

$c \leftarrow c \times 2$

write(C, c)

$a \leftarrow a + b + c$

write(A, a)

T_2

read(A, a)

$a \leftarrow a + 10$

write(A, a)

read(D, d)

$d \leftarrow d - 10$

read(E, e)

read(B, b)

$e \leftarrow e + b$

write(E, e)

$d \leftarrow d + e$

write(D, d)

How does the log look like? [TID, DID, old, new]

Outline

- 7 Recovery
 - Log-based recovery
 - Checkpoints
 - No-undo/no-redo algorithm

Checkpoints

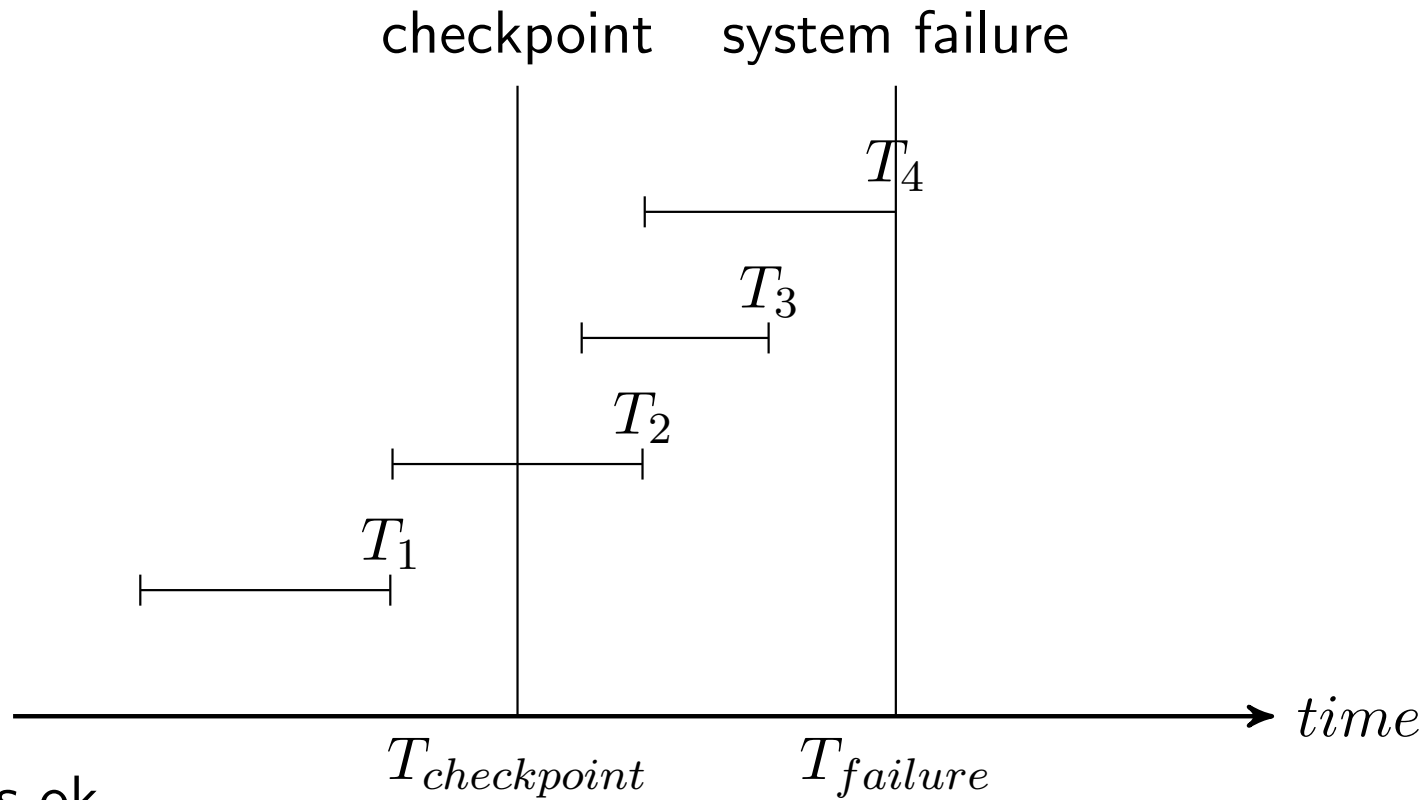
How big can a log file become?

How long would recovery run?

Checkpoints define a position in the log that can be used to start recovery from

checkpoint $\neq$ savepoint!!!!

Idea



- T_1 is ok
- T_2 and T_3 are redone
- T_4 is undone

Checkpointing

- Checkpointing speeds up recovery by flushing dirty pages to disk
- During execution periodically perform checkpointing:
 - 1 Output the log buffers to the log
 - 2 Force database buffers to the disk
 - 3 Add checkpoint record to the log (along with the list of active transactions)
- During recovery
 - Undo all transactions that have not committed
 - Redo all transactions that have committed after checkpoint

Outline

- 7 Recovery
 - Log-based recovery
 - Checkpoints
 - No-undo/no-redo algorithm

No-undo/no-redo algorithm

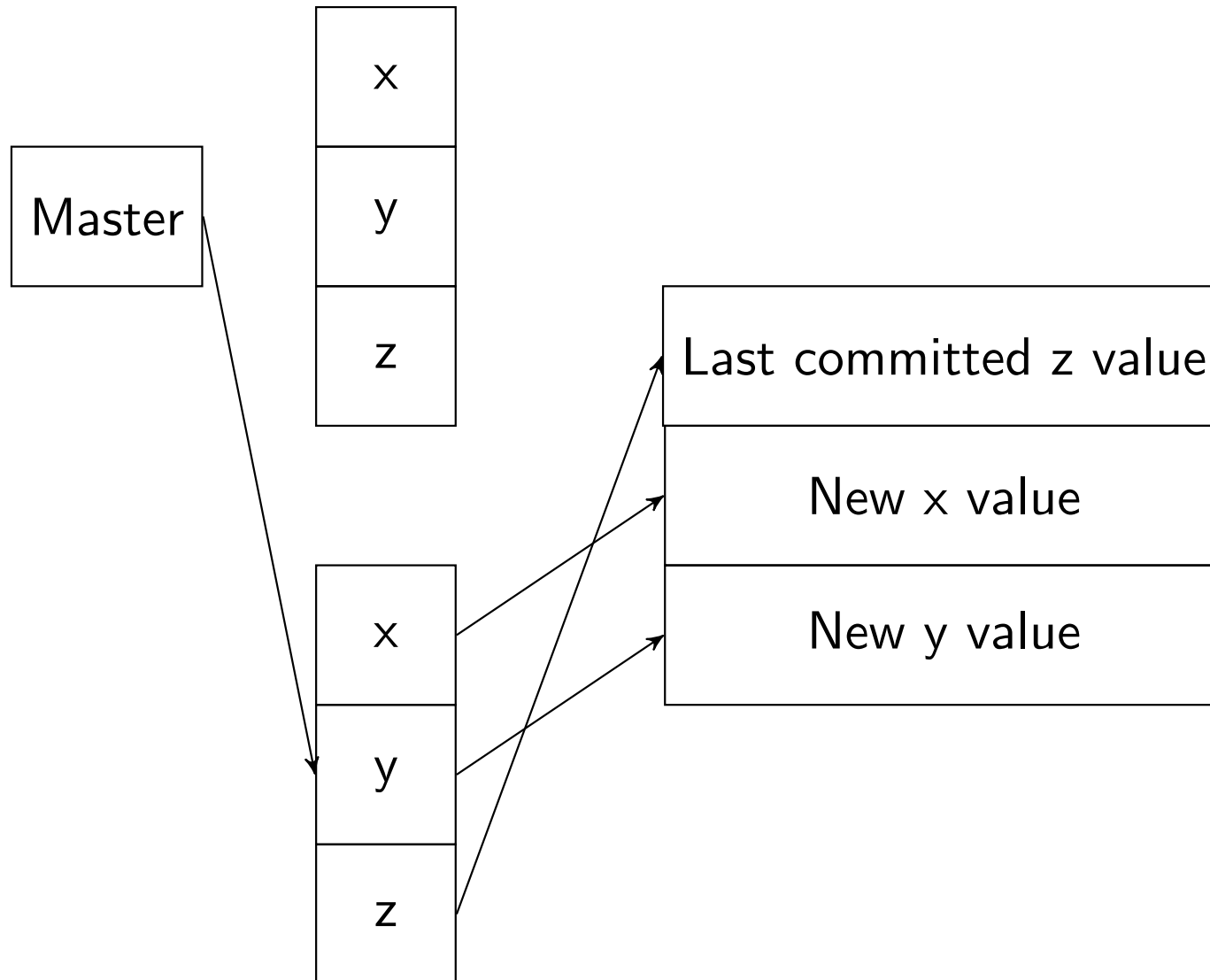
Algorithm

- No-undo
do not change the database during a transaction
- No-redo
on commit, write changes to the database in a single atomic action
- No logs required

Advantages

- Recovery is instantaneous
- No recovery code needs be written
- Atomic writes of many pages is accomplished by using the **shadow paging** technique

Example



No-undo/no-redo algorithm

Advantages

- Commit can be realized efficiently
- Restart is very fast: one disk read

Problems

- Access to stable storage is indirect
- Garbage collection of stable storage is required
- Original layout of data is destroyed
- Concurrent transactions are difficult to support